



**UNIVERSIDAD TECNOLÓGICA DE
ACAPULCO**



**INGENIERÍA EN DESARROLLO Y
GESTIÓN DE SOFTWARE**

**MATERIA: SEGURIDAD EN EL
DESARROLLO DE APLICACIONES**

GRADO: 8

GRUPO: A

**202204037 Jonathan yahir gatica
salanueva**

DOCENTE: PABLO HIGUERA MARIANO

ACTIVIDAD: PORTAFOLIO

**ACAPULCO, GRO. 28 DE MARZO DE
2025**

Índice

Introducción	3
Desarrollo	4
Portada de equipo :	4
Investigación sobre el CMMI (Modelo de Madurez de la Capacidad Integrado)	5
exposición sobre inyecciones sql :	7
Conclusión	8
TERCER PARCIAL	9
Introducción:.....	10
Describe los métodos de seguridad que se utilizaron.	11
Protección contra ataques de fuerza bruta con Axes.	11
.....	11
Implementación de autenticación multifactor (MFA) con Django OTP.....	12
Hashing seguro de contraseñas.	13
Validación de contraseñas seguras.....	14
Middleware de seguridad.....	14
.....	14
Protección de caché con Redis.....	15
Protección contra ataques CSRF.....	15
Protección contra Clickjacking.	15
Expiración del enlace de recuperación de contraseña.....	16
Views :	17
Protección de rutas con autenticación (@login_required)	17
Protección de accesos con permisos de administrador (@user_passes_test).....	18
.....	18
Protección contra ataques de fuerza bruta con Rate Limiting.....	18
Protección contra cuentas bloqueadas.....	19
Verificación de correo antes del registro.....	19
Protección contra creación de cuentas con datos duplicados	20
Protección en la gestión de productos (solo permite al usuario modificar sus propios productos).....	20
Protección contra eliminación de productos de otros usuarios	21
Conclusión	22

Introducción

En el mundo de la tecnología y la ciberseguridad, la eficiencia y la seguridad de los sistemas informáticos juegan un papel fundamental en el desarrollo de software y la gestión empresarial. En este trabajo, se exploran dos temas clave: el **CMMI (Capability Maturity Model Integration)** y las **inyecciones SQL basadas en errores**.

Por un lado, el CMMI es un modelo de madurez que permite a las organizaciones mejorar la calidad y eficiencia de sus procesos mediante una estructura escalonada de evolución. Su implementación ayuda a las empresas a optimizar sus operaciones y alcanzar altos estándares de competitividad.

Por otro lado, la ciberseguridad es un aspecto crítico en el desarrollo de software, y una de las amenazas más comunes son las **inyecciones SQL basadas en errores**. Este tipo de ataque explota vulnerabilidades en aplicaciones web para obtener información sensible de la base de datos, poniendo en riesgo la integridad de los sistemas.

A lo largo de este trabajo, se profundiza en la importancia del CMMI en la mejora de procesos organizacionales y en los riesgos asociados a las inyecciones SQL, junto con las estrategias de prevención necesarias para mitigar sus efectos.

Desarrollo:

Portada de equipo:

UNIVERSIDAD TECNOLÓGICA DE ACAPULCO



REPRESENTANTE: MARQUEZ FLORES JOSE GULMARO 202204027

EQUIPO:

- REAL GAMBOA EMANUEL 202204082
- ALEMAN TORRES JASON 202204008
- GATICA SALANUEVA JONATHAN YAHIR 202204037
- JOSUÉ RAMSÉS ROJAS VALLEJO 202204024

MATERIA: SEGURIDAD EN EL DESARROLLO
DE APLICACION.

MTRO. PABLO HIGUERA MARIANO

TEMA: FORMACION DE EQUIPOS.

Investigación sobre el CMMI (Modelo de Madurez de la Capacidad Integrado)

Introducción al CMMI

El CMMI (Capability Maturity Model Integration) es un modelo de madurez utilizado para mejorar los procesos dentro de las organizaciones, enfocándose en una optimización continua y un control más efectivo de sus actividades. Desarrollado por el Software Engineering Institute (SEI) de la Universidad Carnegie Mellon, el CMMI se centra en la mejora de procesos clave en áreas como el desarrollo de software, mantenimiento y gestión de proyectos. Este modelo permite a las organizaciones evaluar la madurez de sus procesos y planificar mejoras sistemáticas con el objetivo de alcanzar estándares más altos de calidad y eficiencia.

Objetivo del CMMI

El principal objetivo del CMMI es proporcionar una estructura detallada para medir la madurez y la capacidad de los procesos dentro de una organización. Con esto, se busca mejorar la eficiencia, la calidad de los productos o servicios, y fomentar la capacidad de innovación a través de la mejora continua. El modelo está diseñado para ayudar a las empresas a gestionar sus procesos de manera más eficaz, lo que, a su vez, incrementa su competitividad en el mercado.

Niveles de Madurez del CMMI

El CMMI está compuesto por cinco niveles de madurez, que describen cómo los procesos de una organización pueden evolucionar y mejorar con el tiempo.

El **Nivel 1 (Inicial)** se caracteriza por procesos impredecibles y desorganizados, donde la falta de estructura depende en gran medida de las habilidades individuales.

El **Nivel 2 (Gestionado)** representa el momento en que la organización comienza a gestionar sus procesos de forma más efectiva, implementando prácticas de planificación y control de calidad.

El **Nivel 3 (Definido)** es alcanzado cuando los procesos se estandarizan, se documentan y se comunican a todos los miembros de la organización, con un enfoque

en la mejora continua.

En el Nivel 4 (Cuantitativamente Gestionado), los procesos son medidos y controlados a través de datos estadísticos, que son utilizados para mejorar el rendimiento y predecir los resultados futuros.

Finalmente, el **Nivel 5 (Optimizado)** es el nivel más alto, en el que la organización se enfoca en la mejora continua y la innovación, utilizando datos y aprendizaje organizacional para optimizar los procesos.

Áreas Clave del CMMI

El CMMI cubre diversas áreas clave que son fundamentales para la mejora de los procesos dentro de una organización. Algunas de las más relevantes incluyen la gestión de proyectos, que abarca la planificación, seguimiento y control de proyectos; el desarrollo de productos, relacionado con actividades de diseño, desarrollo y pruebas de productos; la gestión de la calidad, que promueve la implementación de prácticas de calidad en todos los procesos; y la gestión de recursos humanos, que se centra en mejorar las habilidades y competencias del personal.

Beneficios del CMMI

Implementar el CMMI en una organización trae consigo una serie de beneficios importantes. Uno de los principales es la mejora de la calidad de los productos o servicios, al asegurar que todos los procesos estén alineados con estándares de alta calidad. Además, permite una reducción de los costos operativos al optimizar los procesos y eliminar ineficiencias. El CMMI también contribuye a una mayor satisfacción del cliente, ya que ofrece productos y servicios más confiables y de mejor calidad, lo que fortalece la relación con los clientes. Por último, impulsa un incremento de la eficiencia organizacional, facilitando una mejor gestión de los recursos y procesos dentro de la empresa.

exposición sobre inyecciones sql:

INYECCIÓN SQL BASADA EN ERRORES (EJEMPLO FALSO)

01 Se basa en provocar errores en la base de datos para obtener información útil sobre la misma

`www.ejemplo.com/web.php? Id=`

Debido a que es un caracter SQL y esto es vulnerable a la inyección SQL esa comilla simple se convertira en parte de la consulta. Se rompe la consulta en el backend y se produce un error

02

`www.ejemplo.com/web.php? Id=' OR 1=1 --'`

aparece un error en la base de datos y este nos muestra la versión que sirve para poder crear más query malisiosos

En esta presentación me toco hablar sobre las inyecciones sql basada en errores, esta nos habla sobre un atacante que intenta provocar errores a la propia base de datos mediante a sentencias sql una vez el atacante ingrese la inyección por ejemplo en la url de la página esta se romperá en el backen, provocando un error en la base de datos que mandara un mensaje como la versión de esta entre otras cosas, que pueden ayudar al atacante para crear más query maliciosos.

Conclusión

La gestión eficiente de los procesos y la seguridad informática son dos pilares fundamentales en cualquier organización que utilice tecnología en sus operaciones. El CMMI, al proporcionar un marco estructurado para evaluar y mejorar la madurez de los procesos, permite a las empresas optimizar sus recursos y elevar la calidad de sus productos y servicios. Su aplicación no solo mejora la eficiencia interna, sino que también incrementa la competitividad en el mercado.

Por otro lado, la seguridad en el desarrollo de software es un aspecto crítico que no puede ser ignorado. Las inyecciones SQL basadas en errores representan una amenaza significativa para las aplicaciones web, ya que pueden exponer datos confidenciales y comprometer la integridad de los sistemas. Sin embargo, mediante la implementación de buenas prácticas de programación, como el uso de consultas parametrizadas, gestión adecuada de errores y restricciones de permisos, es posible reducir drásticamente el riesgo de estos ataques.

En conclusión, tanto el CMMI como la ciberseguridad son aspectos esenciales en el desarrollo y la gestión de software. Mientras que el CMMI ayuda a optimizar procesos y mejorar la calidad, la seguridad informática protege estos avances, asegurando que los sistemas sean confiables y resistentes ante amenazas cibernéticas. Implementar ambos enfoques de manera adecuada permite a las organizaciones operar con mayor eficiencia, seguridad y confianza en un entorno cada vez más digitalizado.

TERCER PARCIAL

Introducción:

En el desarrollo de nuestro sistema de gestión de almacén, la seguridad fue una prioridad fundamental para garantizar la protección de los datos y la confiabilidad del sistema. Implementamos diversas estrategias de seguridad para prevenir vulnerabilidades y reforzar la autenticación de usuarios, asegurando un entorno seguro para la administración de inventarios. A continuación, se detallan las principales medidas adoptadas para mitigar riesgos y fortalecer la integridad de la aplicación.

Describe los métodos de seguridad que se utilizaron.

Para esta actividad, nuestro equipo creó un sistema de gestión de almacenes empleando Django como framework backend y desplegándolo en Railway. Esta plataforma facilita la administración de inventarios, la gestión de usuarios y el control de operaciones dentro del almacén de manera óptima. Como parte de nuestra estrategia de seguridad, incorporamos múltiples medidas de protección para fortalecer la aplicación frente a posibles ataques y vulnerabilidades.

Protección contra ataques de fuerza bruta con Axes.

```
INSTALLED_APPS = [  
    # 'admin_soft.apps.AdminSoftDashboardCo  
    "django.contrib.staticfiles", # Asegú  
    'jazzmin',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'whitenoise.runserver_nostatic',  
    'account',  
    'axes', # Correcto  
    'django_otp',
```

```
AUTHENTICATION_BACKENDS = (  
    'axes.backends.AxesBackend', # Backend de Ax  
    'django.contrib.auth.backends.ModelBackend',  
)  
  
AXES_FAILURE_LIMIT = 5 # Número máximo de intent  
AXES_COOLOFF_TIME = 1 # Tiempo en horas antes de  
AXES_LOCKOUT_TEMPLATE = 'lockout.html'  
AXES_ONLY_USER_FAILURES = True # Solo bloquear a
```

Axes es una librería que protege contra ataques de fuerza bruta bloqueando usuarios después de varios intentos fallidos.

- **Límite de intentos fallidos:** Se permite un máximo de 5 intentos antes de bloquear la cuenta.
- **Tiempo de bloqueo:** La restricción dura 1 hora antes de restablecer el acceso automáticamente.
- **Mensaje de bloqueo personalizado:** Se muestra una plantilla específica (lockout.html) al usuario afectado.
- **Bloqueo individual:** Solo se restringe la cuenta del usuario, sin afectar toda la IP.

Implementación de autenticación multifactor (MFA) con Django OTP.

```
INSTALLED_APPS = [  
    # 'admin_soft.apps.AdminSoftDashboard',  
    'django.contrib.staticfiles', # As  
    'jazzmin',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'whitenoise.runserver_nostatic',  
    'account',  
    'axes', # Correcto  
    'django_otp',  
    'django_otp.plugins.otp_totp',  
    'debug_toolbar',  
]
```

```
MIDDLEWARE = [  
    "debug_toolbar.middleware.DebugToolbarMiddleware",  
    'axes.middleware.AxesMiddleware',  
    'django.middleware.security.SecurityMiddleware',  
    'whitenoise.middleware.WhiteNoiseMiddleware',  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    'django.middleware.csrf.CsrfViewMiddleware',  
    'django.contrib.auth.middleware.AuthenticationMiddleware',  
    'django.contrib.messages.middleware.MessageMiddleware',  
    'django.middleware.clickjacking.XFrameOptionsMiddleware',  
    'django_otp.middleware.OTPMiddleware',  
]
```

- **Autenticación en dos pasos (MFA):** Añade una capa extra de seguridad mediante códigos de un solo uso.
- **Generación de códigos temporales:** `django_otp.plugins.otp_totp` crea códigos que cambian periódicamente.
- **Protección de acceso:** `OTPMiddleware` obliga a los usuarios a verificar un segundo factor antes de ingresar a secciones sensibles.

Hashing seguro de contraseñas.

```
PASSWORD_HASHERS = [  
    'django.contrib.auth.hashers.PBKDF2PasswordHasher',  
    'django.contrib.auth.hashers.Argon2PasswordHasher',  
    'django.contrib.auth.hashers.BCryptSHA256PasswordHasher',  
]
```

Almacenamiento seguro de contraseñas: Utiliza técnicas de cifrado avanzadas para proteger credenciales.

- **PBKDF2:** Algoritmo lento y resistente contra ataques de fuerza bruta.
- **Argon2:** Ofrece mayor seguridad frente a ataques con hardware especializado.
- **BCryptSHA256:** Alternativa más segura en comparación con el hashing tradicional SHA256.

Validación de contraseñas seguras.

```
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
        'OPTIONS': {'min_length': 12}, # Exige al menos 12 caracteres
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

- **Restricción de contraseñas débiles:** Impide el uso de claves fáciles de adivinar.
- **Longitud mínima segura:** Requiere al menos 12 caracteres para reducir el riesgo de ataques.

Middleware de seguridad.

```
'django.contrib.sessions.middleware.SessionMiddleware',
```

Activas protecciones integradas de Django, como:

- **Seguridad HSTS:** Fuerza el uso de HTTPS en producción para evitar ataques de interceptación.
- **Control de archivos subidos:** Restringe la carga de archivos potencialmente maliciosos.

Protección de caché con Redis.

```
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.redis.RedisCache',
        'LOCATION': os.environ.get('REDIS_URL', 'redis://127.0.0.1:6379'),
        'OPTIONS': {
            'CLIENT_CLASS': 'django_redis.client.DefaultClient',
        }
    }
}
```

- Uso de Redis para caché: Sustituye el almacenamiento en memoria para una gestión más eficiente.
- Mayor seguridad: Previene la exposición prolongada de datos sensibles en la caché.

Protección contra ataques CSRF.

```
'django.middleware.csrf.CsrfViewMiddleware',
```

- **Defensa contra CSRF:** Evita ataques que explotan la confianza del usuario en la aplicación.
- **Validación de solicitudes:** Garantiza que solo peticiones legítimas sean aceptadas y procesadas.

Protección contra Clickjacking.

```
'django.middleware.clickjacking.XFrameOptionsMiddleware',
```

- **Protección contra iframes maliciosos:** Impide que la aplicación se incruste en sitios externos no autorizados.
- **Prevención de ataques de Clickjacking:** Evita que los usuarios sean engañados para interactuar con elementos ocultos.

Expiración del enlace de recuperación de contraseña.

```
PASSWORD_RESET_TIMEOUT=14400
```

- Los enlaces de recuperación de contraseña **caducan después de 4 horas**, reduciendo el riesgo de ser explotados.

Views:

Protección de rutas con autenticación (@login_required)

```
from django.contrib.auth.decorators import login_required,
```

```
# logout
@login_required
def logout_view(request):
    logout(request)
    return redirect('signin')

# Index
@login_required
def index(request):
    return render(request, 'index.html')
```

```
@login_required
def profile(request):
    return render(request, 'profile.html')

@login_required
def manage_product(request):
    products = Product.objects.filter(user=request.user)
    categories = Category.objects.all()
    branches = Branch.objects.all()
    return render(request, 'manage_product.html', {'products': products, 'categories': categories, 'branches': branches})

@login_required
def product_list(request):
    products = Product.objects.all()
    return render(request, 'product.html', {'products': products})
```

- **Restricción de acceso:** Evita que usuarios no autenticados ingresen a rutas protegidas.
- **Redirección segura:** Envía al usuario a la página de inicio de sesión si no ha iniciado sesión previamente.

Protección de accesos con permisos de administrador (@user_passes_test)

```
@user_passes_test(is_admin)
def card(request):
    return render(request, 'sidebar.html')

@user_passes_test(is_admin)
def mode(request):
    return render(request, 'mode.html')
```

```
# Función que verifica si el usuario es admin o staff
def is_admin(user):
    return user.groups.filter(name='Admin').exists() or user.is_staff

# Vista para redirigir solo a usuarios con permisos a /admin/
@user_passes_test(is_admin, login_url='/error403/')
def custom_admin_redirect(request):
    return redirect('/admin/') #django-admin
```

- **Acceso restringido:** Solo los administradores o usuarios con privilegios pueden ingresar a ciertas rutas.
- **Protección de vistas sensibles:** Limita el acceso a /admin/ y otras secciones administrativas.

Protección contra ataques de fuerza bruta con Rate Limiting

```
def signin(request):
    if request.user.is_authenticated:
        if request.user.is_staff:
            return redirect('/admin/') # Redirige a Django Admin si es administrador
            return redirect('/') # Redirige a home si es usuario normal

    if request.method == "POST":
        email = request.POST.get("email")
        password = request.POST.get("password")

        try:
            user = User.objects.filter(email=email).first()

            if user:
                # * Bloqueo manual: Verificar si el usuario está activo
                if not user.is_active:
                    messages.error(request, "Tu cuenta ha sido bloqueada por un administrador.")
                    return redirect('signin')

                user = authenticate(request, username=user.username, password=password)

                if user:
                    login(request, user)
                    return redirect('/admin/' if user.is_staff else '/')
                else:
                    messages.error(request, "Nombre de usuario o contraseña incorrectos.")
            else:
                messages.error(request, "Nombre de usuario o contraseña incorrectos.")

        except Exception as e:
            messages.error(request, "Ocurrió un error. Intenta nuevamente.")
            print(f"Error de autenticación: {e}")

    return render(request, "signin.html")
```

- **Limitación de intentos:** Restringe los intentos de inicio de sesión a cinco por minuto por dirección IP.
- **Defensa contra fuerza bruta:** Previene accesos no autorizados mediante intentos repetidos de autenticación.

Protección contra cuentas bloqueadas

```
if user:
    # ♦ Bloqueo manual: Verificar si el usuario está activo
    if not user.is_active:
        messages.error(request, "Tu cuenta ha sido bloqueada por un administrador.")
        return redirect('signin')
```

- **Restricción de acceso:** Evita que los usuarios bloqueados puedan iniciar sesión.
- **Protección de cuentas:** Garantiza que las cuentas desactivadas por un administrador no sean utilizadas.

Verificación de correo antes del registro

```
def request_email_verification(request):
    """
    Captura el email y envía un código de verificación.
    """
    if request.method == "POST":
        email = request.POST.get("email").strip().lower()

        # Generar código y guardarlo en la BD
        code = EmailVerification.generate_code()
        EmailVerification.objects.update_or_create(email=email, defaults={"code": code})
        send_verification_email(email, code)

        request.session["pending_email"] = email # Guardar email en sesión
        return redirect("verify_email_code")

    return render(request, "request_email_verification.html")


def verify_email_code(request):
    """
    Verifica el código de verificación y permite continuar con el registro.
    """
    pending_email = request.session.get("pending_email")
    if not pending_email:
        return redirect("request_email_verification")

    if request.method == "POST":
        code = request.POST.get("code")

        verification = EmailVerification.objects.filter(email=pending_email, code=code).first()

        if verification:
            request.session["verified_email"] = pending_email # Guardar email verificado en sesión
            verification.delete() # Eliminar el registro temporal
            return redirect("signup_form")

        messages.error(request, "Código incorrecto.")

    return render(request, "verify_email_code.html")
```

- **Filtrado de cuentas falsas:** Impide el registro de bots y correos no válidos.
- **Verificación obligatoria:** Requiere que los usuarios confirmen su correo antes de completar el registro.

Protección contra creación de cuentas con datos duplicados

```
if User.objects.filter(username=username).exists():
    messages.error(request, "El nombre de usuario ya está en uso.")
    return redirect('signup')

if User.objects.filter(email=email).exists():
    messages.error(request, "El email ya está registrado.")
    return redirect('signup')
```

- **Evita registros duplicados:** Restringe el uso de correos y nombres de usuario ya registrados.
- **Prevención de sobrecarga:** Protege el sistema contra intentos masivos de creación de cuentas repetidas.

Protección en la gestión de productos (solo permite al usuario modificar sus propios productos)

```
@login_required
def edit_product(request, product_id):
    product = get_object_or_404(Product, id=product_id, user=request.user) # Ensure the product belongs to the user

    if request.method == 'POST':
        form = ProductForm(request.POST, request.FILES, instance=product) # Include request.FILES to handle file uploads
        if form.is_valid():
            form.save()
            messages.success(request, 'Product updated successfully.')
            return redirect('manage_product')
    else:
        form = ProductForm(instance=product)

    categories = Category.objects.all()
    branches = Branch.objects.all()

    return render(request, 'account/edit_product.html', {
        'form': form,
        'categories': categories,
        'product': product,
        'branches': branches
    })
```

- **Control de edición:** Los usuarios solo pueden modificar los productos que les pertenecen.

- **Restricción de acceso:** Se impide que un usuario visualice o edite productos de otros.

Protección contra eliminación de productos de otros usuarios

```
@login_required
def delete_product(request, product_id):
    product = get_object_or_404(Product, id=product_id, user=request.user) # Ensure the product belongs to the user
    if request.method == 'POST':
        product.delete()
        #messages.success(request, 'Product deleted successfully.')
        return redirect('manage_product') # Ensure it redirects to manage_product
    return redirect('manage_product') # Redirect to manage_product if not POST
```

- **Restricción de acceso:** Solo los usuarios autenticados pueden eliminar productos.
- **Verificación de propiedad:** Se asegura que el producto pertenezca al usuario antes de eliminarlo.
- **Protección contra solicitudes GET:** Solo permite eliminar productos mediante solicitudes POST.
- **Mensajes de confirmación:** Informa al usuario cuando un producto ha sido eliminado con éxito.
- **Redirección segura:** Después de la eliminación, se redirige a la página de gestión de productos.

Conclusión

La implementación de múltiples capas de seguridad en nuestro sistema garantiza una protección robusta contra ataques cibernéticos y accesos no autorizados. Desde la autenticación multifactorial hasta la prevención de ataques de fuerza bruta y la gestión segura de credenciales, cada medida contribuye a la estabilidad y confiabilidad del sistema. Con estas estrategias, aseguramos una plataforma eficiente y protegida, minimizando vulnerabilidades y fortaleciendo la experiencia de los usuarios.